HOSPITAL QUEUE MANAGEMENT SYSTEM

Micro Project Report

Submitted to the APJ Abdul Kalam University, Kerala
for the partial fulfillment of the requirements for the award of the degree

of

Bachelor of Technology
in
Computer Science and Engineering

By
NAKSHATHRA.D PRN25CS084
SHIFANA FATHIMA.S PRN25CS115
MARJAN.A PRN25CS075
NAYANA B PRN25CS090

Guided by
ANISH A AZIZ , ASSISTANT PROFESSOR , COLLEGE OF ENGINEERING PERUMON

During
OCTOBER 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING PERUMON
CAPE - GOVT. OF KERALA, KOLLAM(D), KERALA

Micro-Project Report

Table of Contents

Chapter

No. Title

1.Introduction
2.Scope of the project
3.Technologies used
4.Problem statement
5.Literature review or background
6.Implementation
7.Details of java classes and architecture
8.Results and Discussion
9.Conclusion
10.Reference

1. Introduction

Hospitals often serve many patients across multiple departments and doctors. If patient turns are managed manually, staff may find it difficult to track token numbers, waiting order, and priority cases. A simple computer-based queue system can make the process more organised and transparent.

The Hospital Patient Token and Queue Management System is a desktop micro-project developed using Java Swing. It provides a graphical user interface (GUI) and connects each stage through Continue and Back actions. It is a prototype intended for academic demonstration rather than a production hospital information system.

1.1Objectives

- Provide a clear, frame-based workflow from the home screen to queue status.
- Allow users to log in and enter basic patient details.
- Allow department and doctor selection.
- Record patient priority and preferred time where provided by the priority screen.
- Generate a token and estimate the waiting time.
- Sort priority cases within the queue for the same department and doctor.
- Display the queue in a table and allow the user to refresh it.

1.2 Project Workflow

Home Frame → Login Frame → Patient Registration Frame → Department Selection Frame →

Doctor Selection Frame → Priority Selection Frame → Token Queue Frame → Queue Status Frame. The Back button returns to the previous relevant frame. The exact navigation may depend on the constructors and action listeners used in the project files.

2. Scope of the Micro-Project

The project focuses on a basic desktop queue workflow. It is suitable for demonstrating Java GUI programming and simple queue management logic.

2.1IncludedFeatures

- Home screen with a route into the application.
- Login screen for a basic sign-in interface.
- Patient registration with name, age, and phone number fields.
- Department selection and doctor selection screens.
- Priority selection, including normal and supported higher-priority categories.
- Token generation with a token number, queue position, and estimated waiting time.
- Queue status table with Update Queue, Back, and Finish controls.
- Doctor-wise and department-wise queue separation.

2.2 Limitations

- Queue data is held in memory and may be lost when the application closes.
- Authentication is a demonstration unless connected to a secure user store.
- Patient details and uploaded proof are not guaranteed to be stored permanently.
- Waiting times are estimates based on a simple fixed interval, not live hospital conditions.
- The application is an educational prototype and must not be used to make clinical decisions.

2.3 Future Enhancements

Possible extensions include database storage, secure authentication, input validation, real-time queue refresh, cancellation handling, audit logs, accessibility improvements, and a configurable estimated service time.

3. Technologies Used

- Java: core programming language and object-oriented structure.
- Java Swing: JFrame, JPanel, JLabel, JTextField, JButton, JTable, JScrollPane, JOptionPane, and other GUI components.
- AWT: layouts, fonts, dimensions, colours, and event-related classes.
- GridLayout, BorderLayout, FlowLayout, and GridBagLayout: arrangement of interface components.
- ArrayList: in-memory collection of patient records.
- Comparator: sorting records by priority and then registration order.
- DefaultTableModel: non-editable data model for the queue table.
- IDE: any Java-compatible IDE such as NetBeans, IntelliJ IDEA, or Eclipse.

The project can be compiled with a standard JDK that supports Swing. No external libraries are required for the described basic interface.

4. Problem Statement

Manual patient queue management can lead to confusion when several departments and doctors are working at the same time. A single shared list may incorrectly mix patients belonging to different doctors. A manual process can also make it harder to identify patients who should receive priority according to the application's configured rules.

The problem addressed by this micro-project is to provide a simple interface that registers a patient, associates the patient with a selected department and doctor, generates a token, and displays the queue in the correct group. Priority sorting must apply only within the same department– doctor pair. A higher-priority patient registered later should appear ahead of normal-priority patients in that same queue, while another doctor's queue remains independent.

4.1ExpectedBehaviour

- A normal patient for Cardiologist A enters that doctor's queue.
- A later higher-priority patient for Cardiologist A is sorted ahead of the normal patient.
- A patient for Cardiologist B remains in a separate queue, even if the department is the same.
- A patient in ENT remains separate from patients in Cardiology.
- Patients with equal priority are ordered by their registration sequence.

5. Literature Review / Background

Queue management systems are commonly used to organise service order in clinics, banks, public offices, and customer support centres. A basic queue typically follows first-in, first-out ordering. In a priority queue, the order can be adjusted using a priority category. A practical appointment system may also separate queues by service counter, department, or provider.

Java Swing is a Java GUI toolkit that provides reusable components and event-driven interaction. JFrame represents a top-level window, JPanel groups components, layout managers position them, and action listeners respond to button clicks. JTable and table models are useful for presenting structured records. ArrayList is suitable for a small in-memory prototype, while Comparator can define a custom ordering.

For this project, the queue combines two ideas: partitioning by department and doctor, then sorting each partition by priority and registration order. This prevents a high-priority patient assigned to one doctor from changing the order of another doctor's queue.

6. Implementation

6.1Home Frame

Purpose: The Home Frame is the first screen shown when the application starts. It introduces the Hospital Patient Token and Queue Management System and provides a clear entry point to the login screen. It can display the project title, a short description, and a Start / Login button.

6.2 Login Frame

Purpose: Provides a login interface before entering patient registration. Typical fields are username and password, with Login and Back buttons Main components: JLabel, JTextField or JPasswordField, JButton, and JOptionPane. In a classroom prototype, validation may be simple. A real system should never store passwords as plain text and should use secure authentication.

6.3 Patient RegistrationFrame

Purpose: Collects basic patient information such as patient name, age, and phone number. Register validates that required fields are completed. Clear empties the fields. Back returns to the preceding screen. Main components: GridBagLayout for aligned labels and text fields; JButton action listeners for Register, Clear, and Back; JOptionPane for validation messages. On successful registration, the application proceeds to department selection or the next configured screen.

6.4 Department SelectionFrame

Purpose: Allows the user to select a hospital department, for example Cardiology or ENT. The selected department is passed to DoctorSelectionFrame so that the available doctors can be filtered or displayed for that department. Main components: department choices using buttons, a combo box, or a list; Continue and Back buttons. The selected department must be retained when moving forward and back.

6.5 Doctor SelectionFrame

Purpose: Displays doctors associated with the chosen department. The selected doctor and department are passed to PrioritySelectionFrame. Main components: doctor selection controls, labels showing the department, and Continue/Back buttons. Doctor selection is important because queue records are grouped by both department and doctor.

6.6PrioritySelectionFrame

Purpose: Allows the patient to select a priority category. The project workflow may include senior citizen, disability, pregnant patient, and normal priority. One option should be mandatory before Continue is allowed. If proof is requested for a category, the interface can provide a file chooser and retain the selected file path. Main components: JRadioButton or JComboBox, ButtonGroup where radio buttons are used, a proof upload button if implemented, a preferred-time selector if implemented, and Continue/Back buttons. The chosen priority, proof path, and preferred time are passed to TokenQueueFrame. Proof selection alone does not verify eligibility.

6.7 Token Queue Frame

Purpose: Collects the patient's name for token generation and displays the generated token, queue position, and estimated waiting time. The current design also displays department, doctor, priority, and preferred time. Main components: JTextField, JLabel, JButton, JPanel, GridBagLayout or BorderLayout, and JOptionPane. Generate Token validates the name, creates a patient record, adds it to the in-memory list, recalculates the relevant queue, and displays the resulting values. Continue opens QueueStatusFrame. Back returns to PrioritySelectionFrame.

Queue algorithm: records are grouped by the combination of department and doctor. Within each group, records are sorted by priority rank and then registration order. In the supplied implementation, pregnant priority has rank 1, disability rank 2, senior priority rank 3, and normal/other priority rank 4. Lower rank values appear first. Each queue is numbered from 1 and waiting time is estimated as 10 minutes multiplied by the zero-based position.

6.8 Queue Status Frame

Purpose: Displays patient and appointment details along with a table of the selected doctor's queue. Update Queue refreshes the displayed rows using the latest in-memory records. Back returns to TokenQueueFrame. Finish closes the status window.

Main components: JTable, DefaultTableModel, JScrollPane, JLabel, JButton, BorderLayout, and GridLayout. Table columns include token, patient name, priority, queue position, and estimated waiting time. Rows are filtered by both department and doctor.

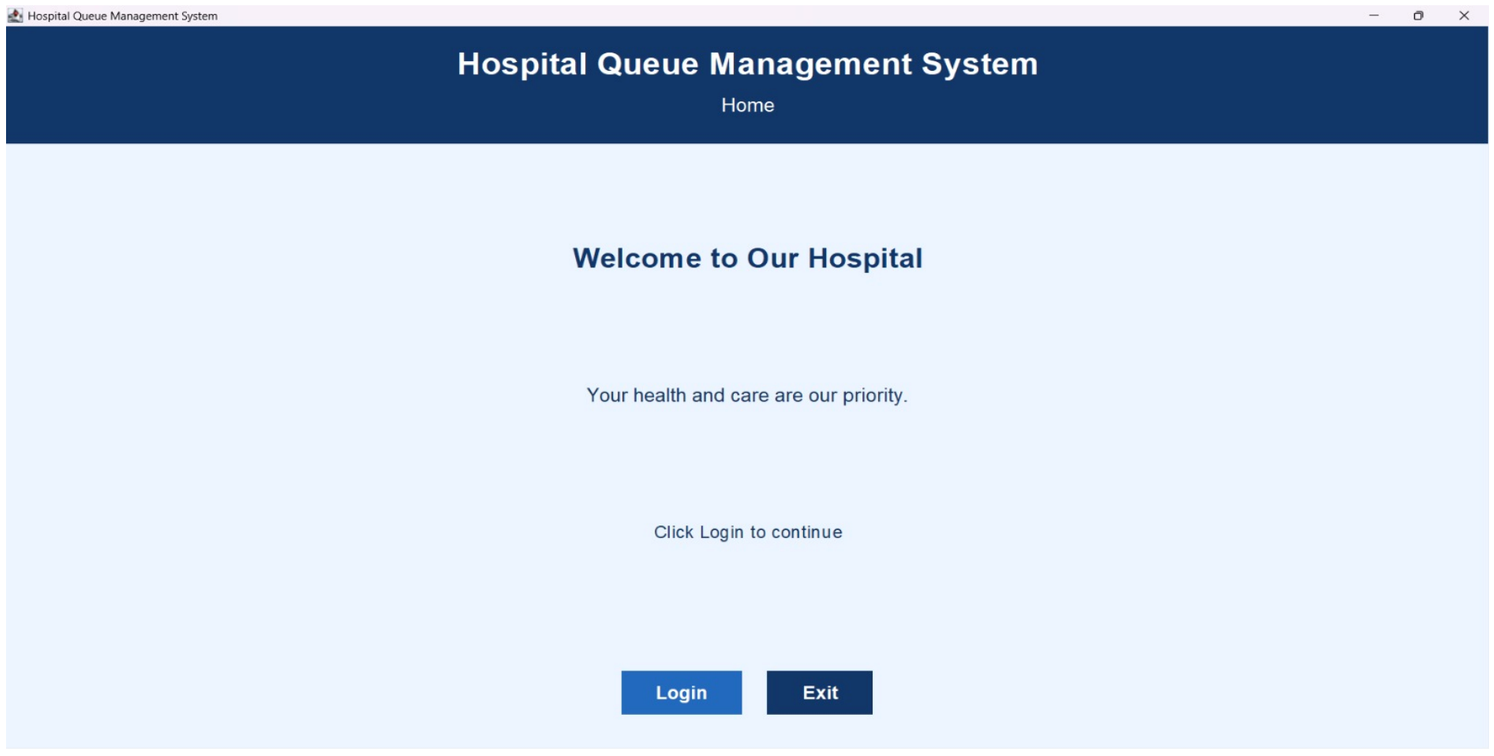
6.9NavigationBetweenFrames

Navigation is implemented through button action listeners. Each frame creates the next frame and passes the selected values through its constructor. The Back button recreates the previous frame with the earlier selections. To preserve the exact state, the previous frame's values should be passed back rather than replaced with empty strings or defaults.

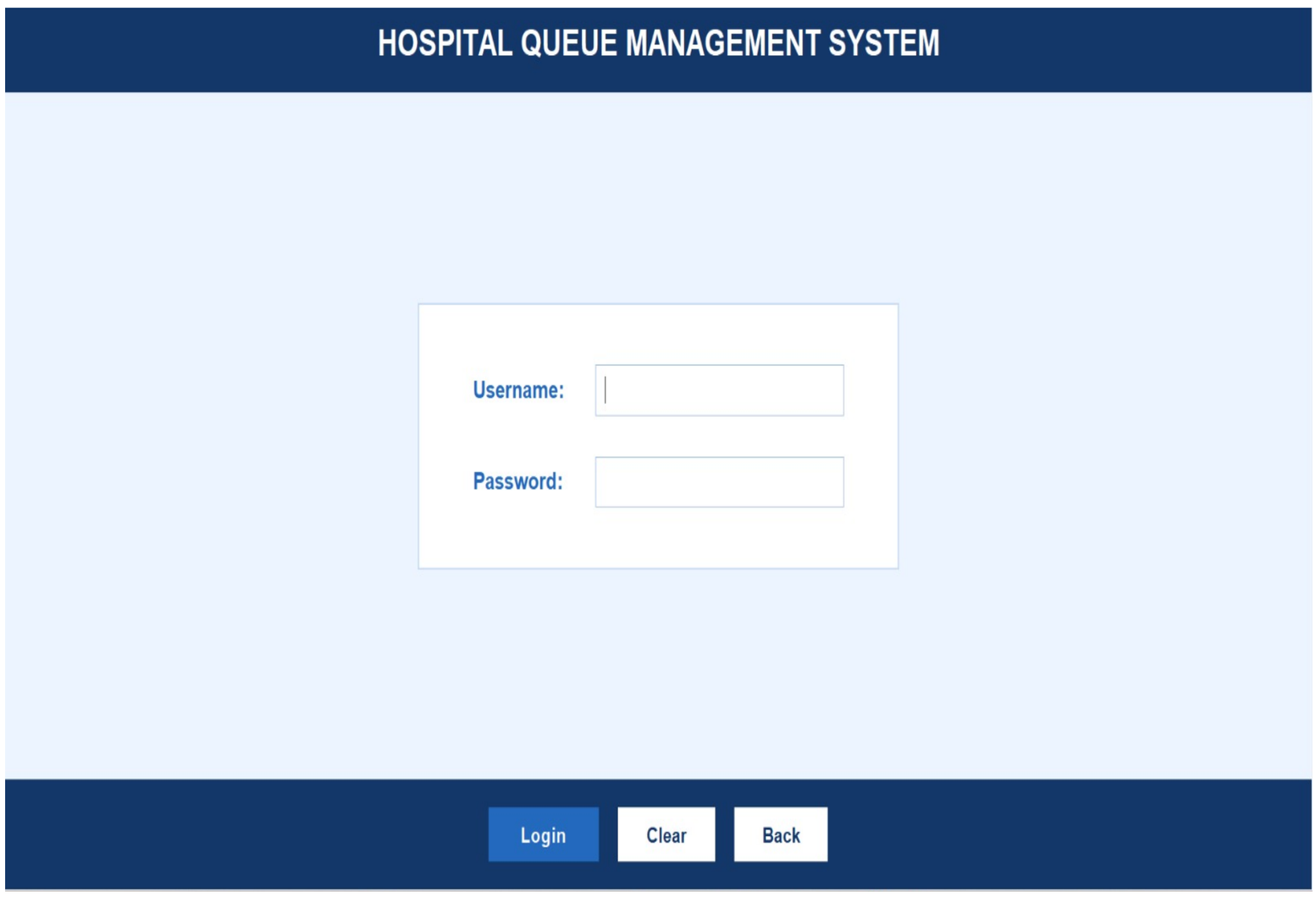
6.10 Queue Sorting Pseudocode

- Add the new patient record to the in-memory patient list.
- Collect the unique department– doctor combinations.
- For each combination, select only the records belonging to that department and doctor.
- Sort the selected records by priority rank, then registration order.
- Assign queue position and token values within that queue.
- Calculate estimated waiting time using the configured interval.
- Refresh the queue table when Update Queue is pressed.

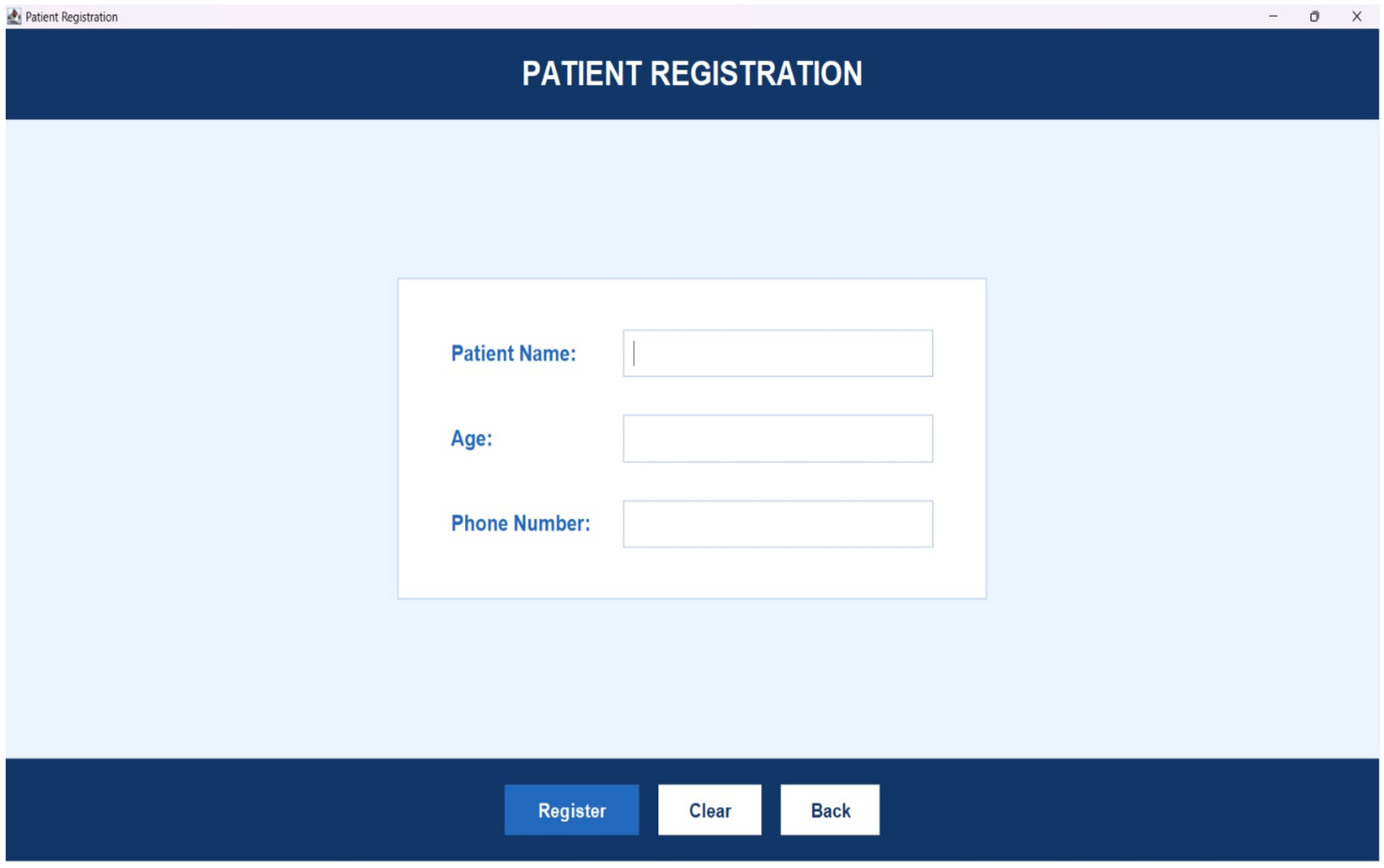
Home frame



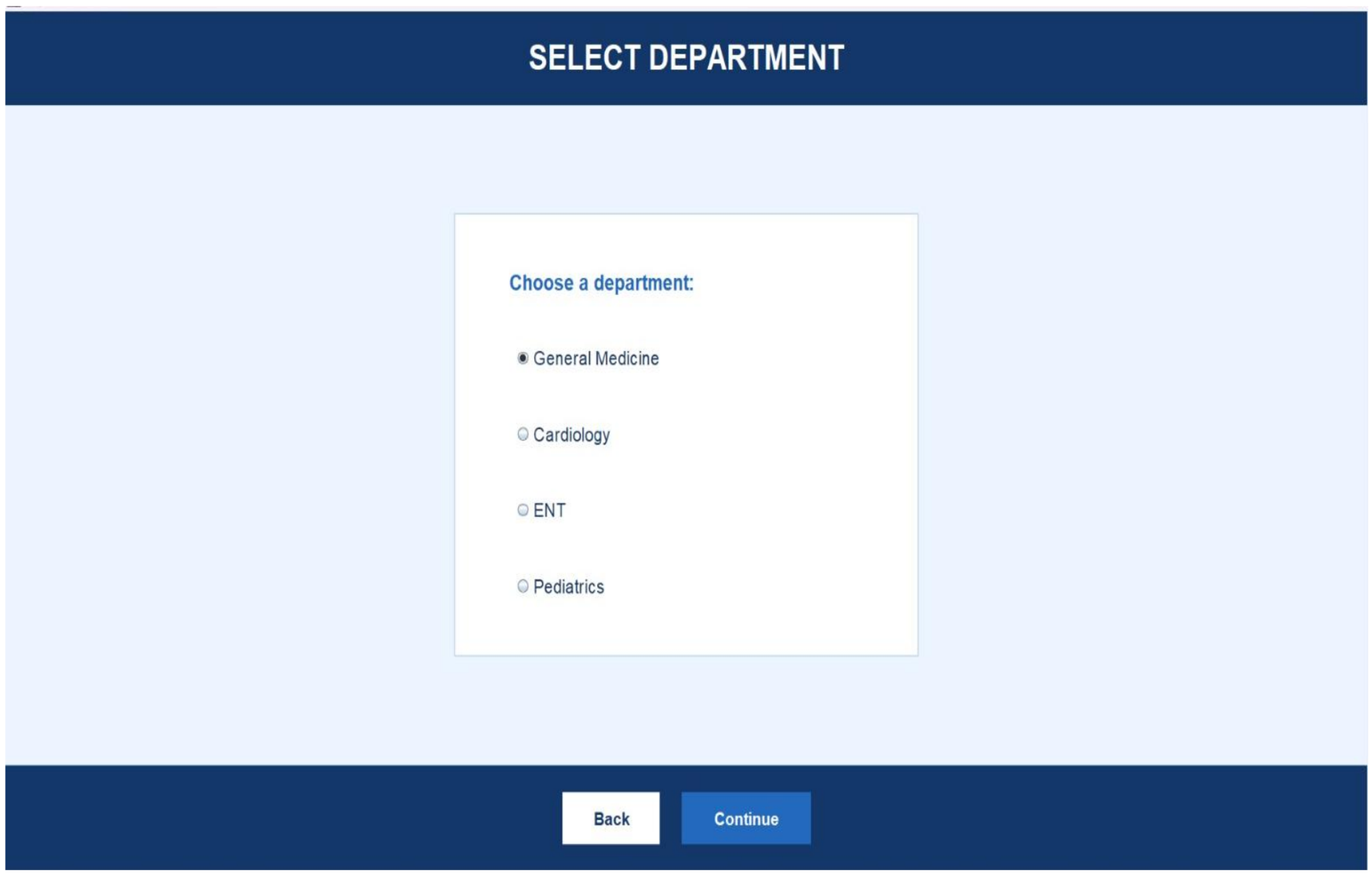
LOGIN FRAME



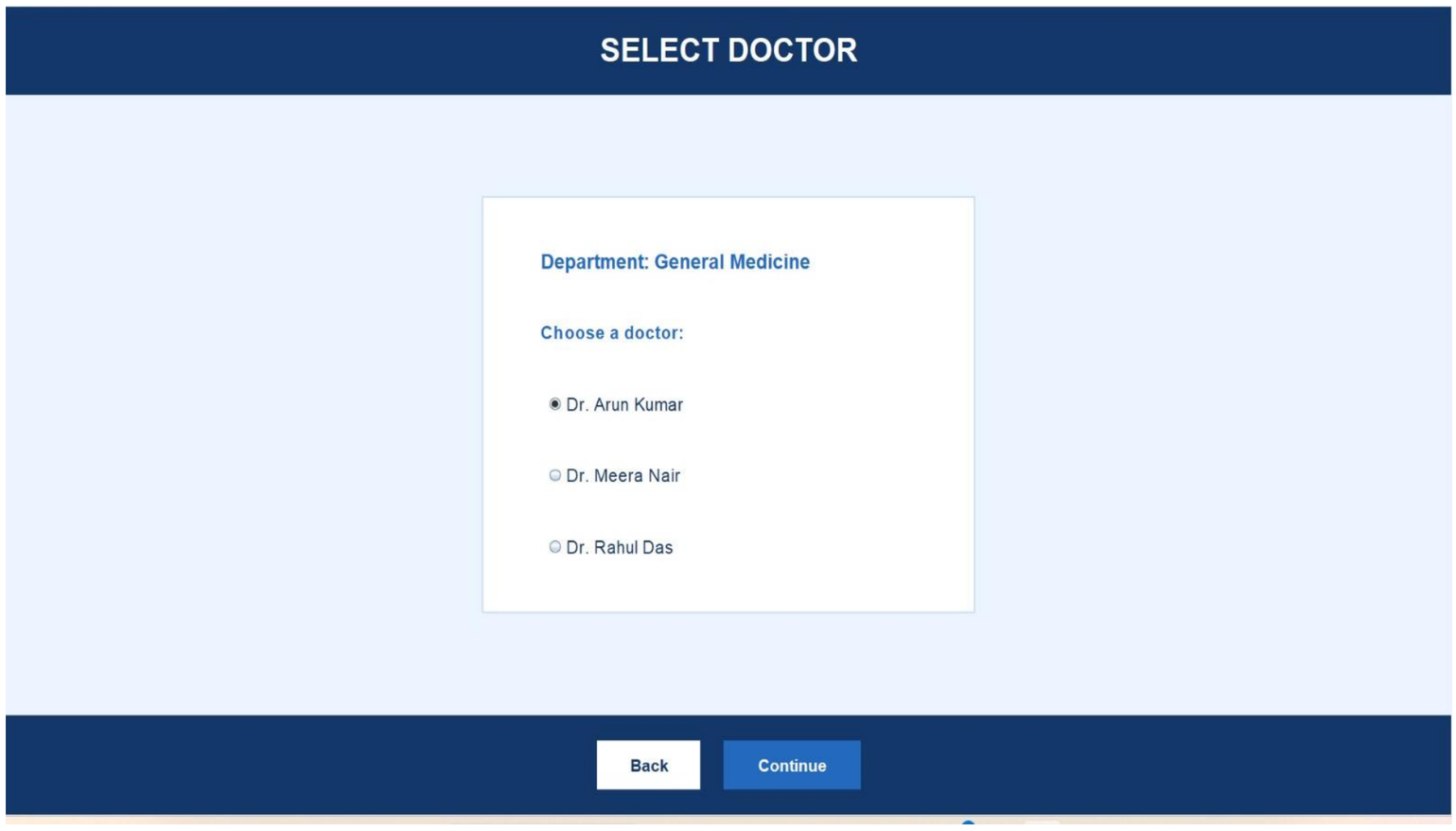
PATIENT REGISTRATION FRAME



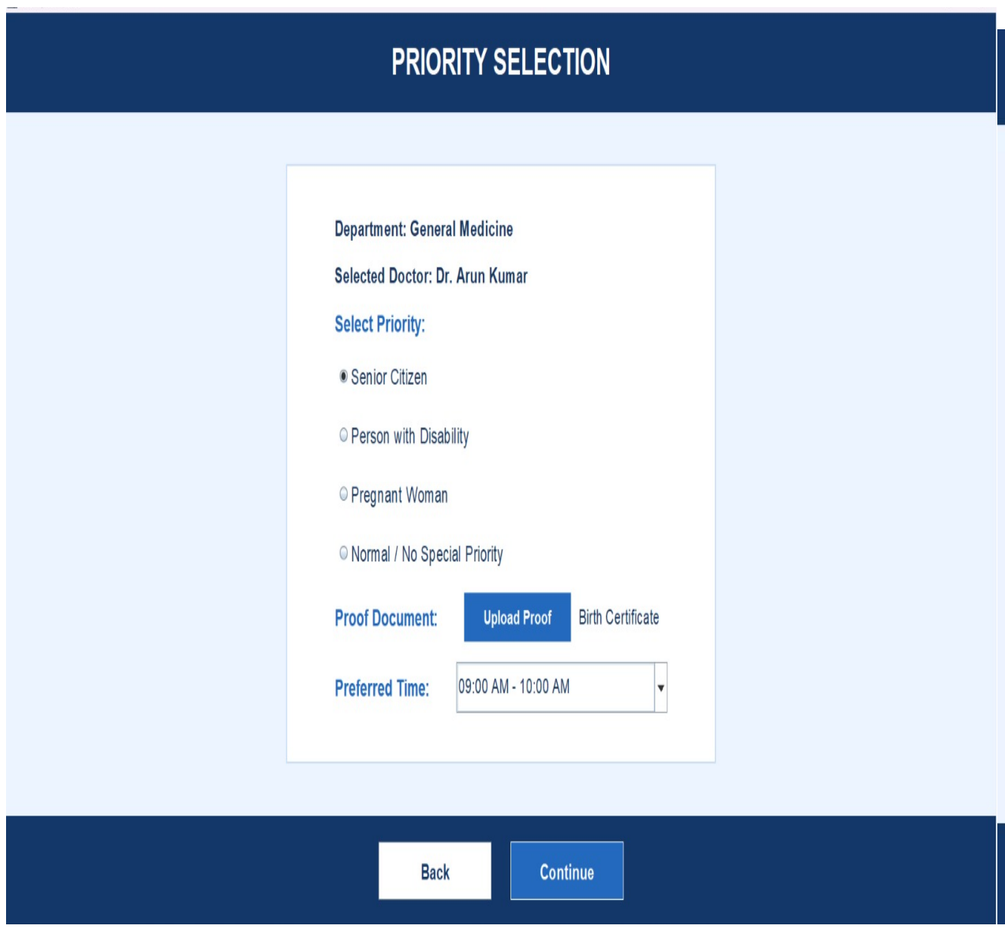
DEPARTMENT SELECTION FRAME

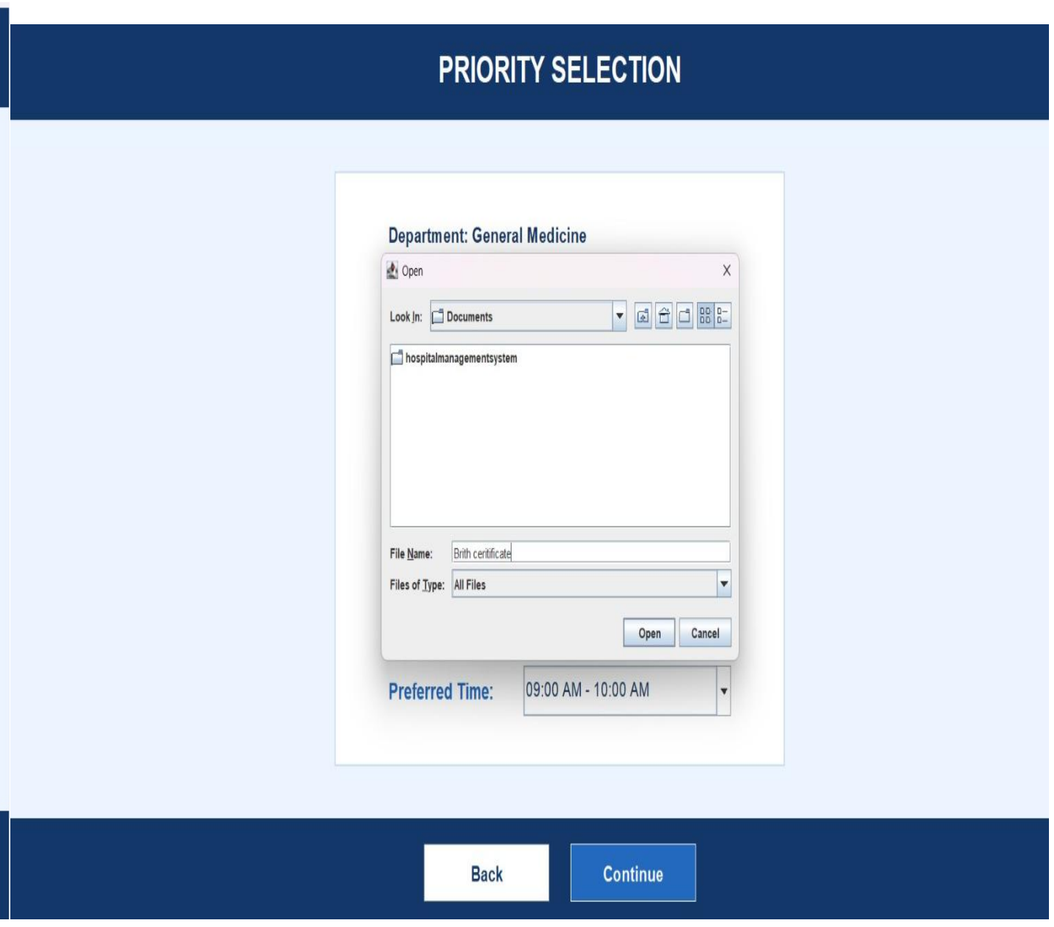


DOCTOR SELECTION FRAME

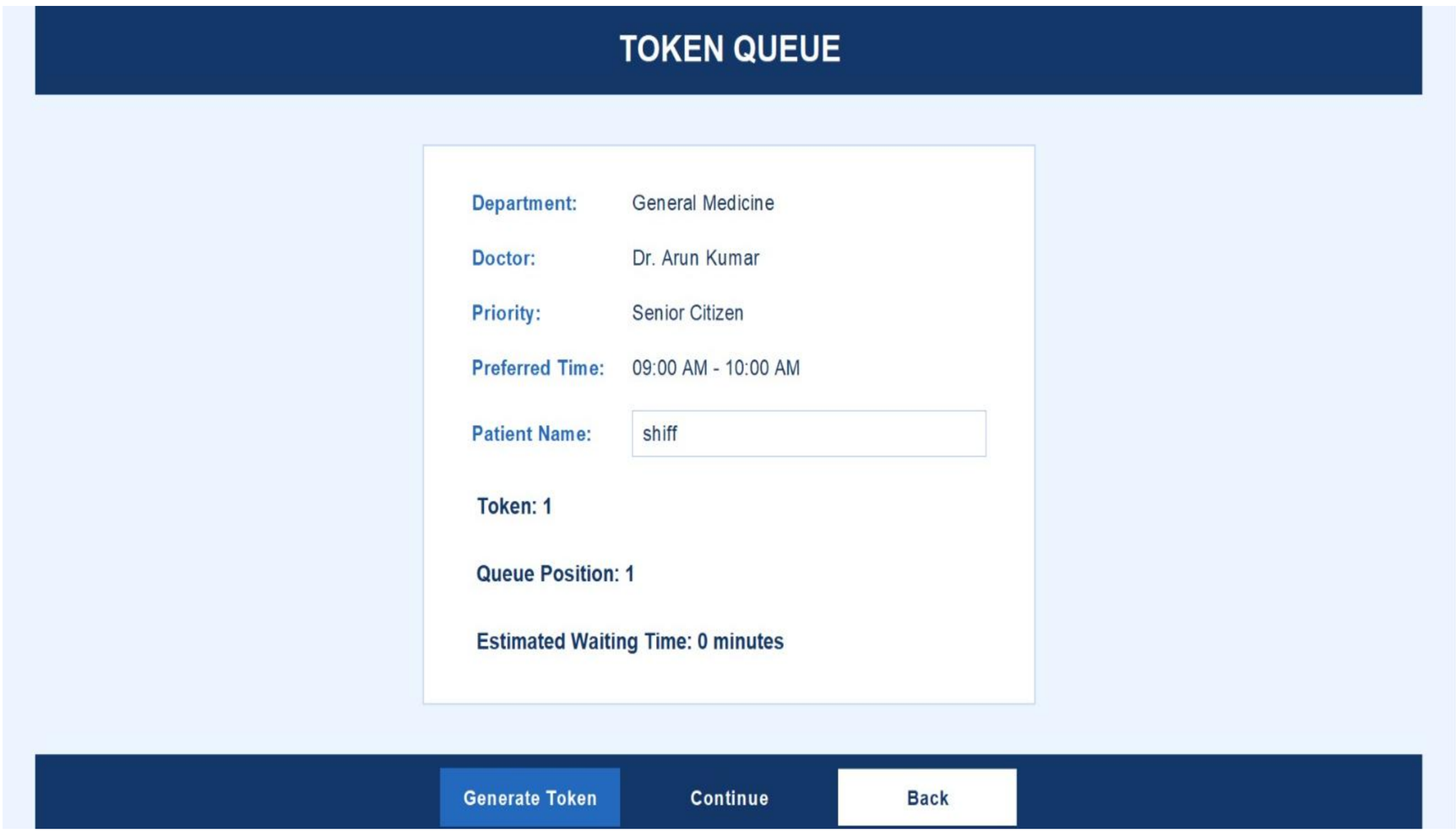


PRIORITY SELECTION FRAME

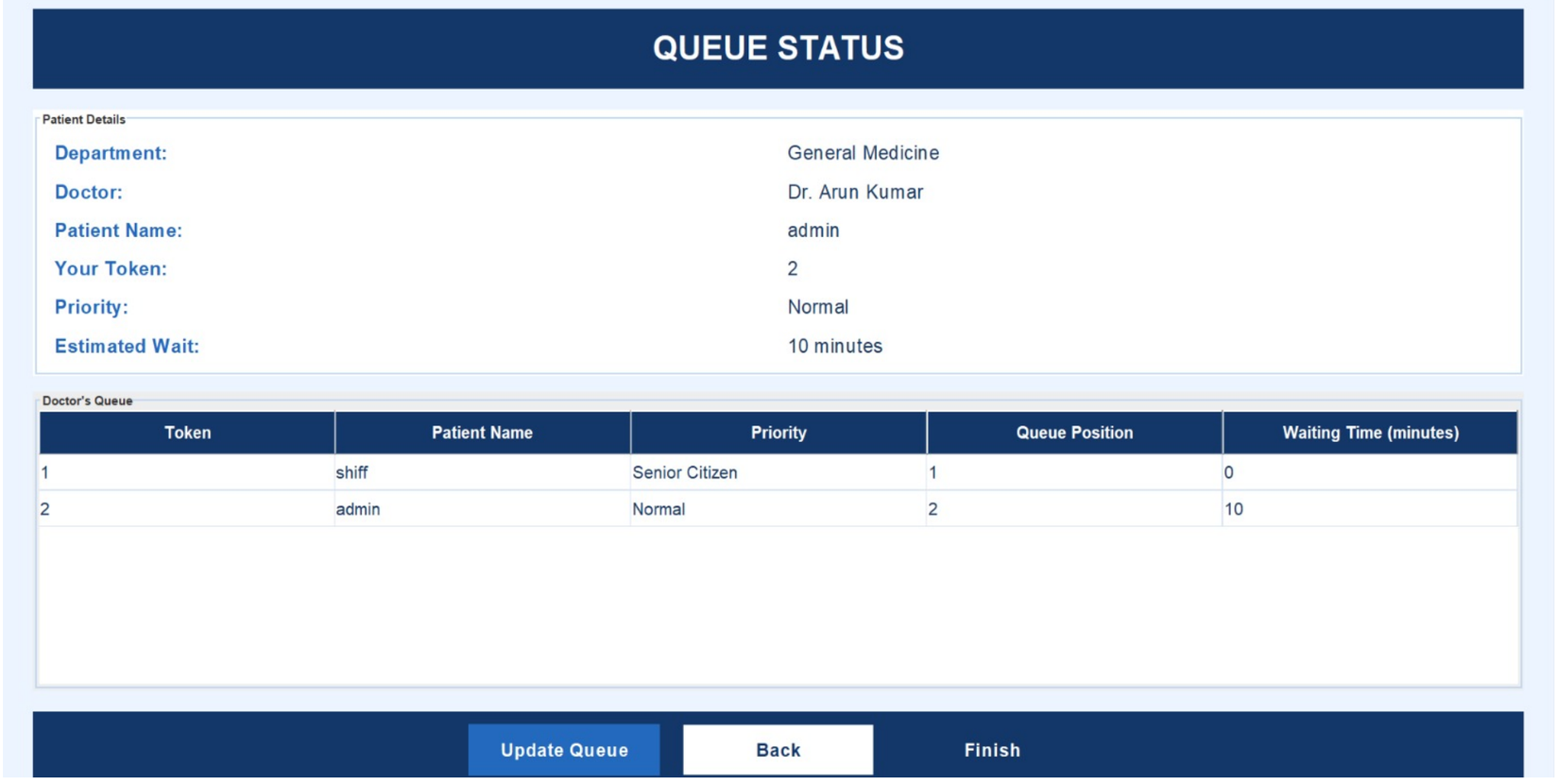




TOKEN QUEUE FRAME



QUEUE STATUS FRAME



7. Details of Java Classes and Architecture

7.1Frame Classes

Home Frame — Initial application screen; directs the user to Login Frame.

Login Frame — Collects login details and routes to patient registration after validation.

Patient Registration Frame — Collects patient name, age, and phone number.

Department Frame / Department Selection Frame — Stores the selected department and opens doctor selection.

Doctor Selection Frame — Stores the selected doctor and passes department and doctor details onward.

Priority Selection Frame — Collects priority, optional proof, and preferred time.

Token Queue Frame — Creates Patient Record objects, maintains the in-memory patient list, sorts queues, and generates token details.

Queue Status Frame — Displays the queue in a JTable and refreshes it on request.

7.2 Patient Record Class

PatientRecord represents a single patient queue entry. Its fields include name, department, doctor, priority, proof, preferredTime, token, queuePosition, waitTime, and registrationOrder. Keeping these values together makes it easier to sort, display, and pass patient data through the queue workflow.

7.3 Architecture

The project follows a simple GUI-driven architecture. Frame classes form the presentation layer. Action listeners connect user actions to validation and navigation. TokenQueueFrame currently contains the queue data and sorting logic, while QueueStatusFrame reads that data and presents it. For a larger system, queue operations could be moved into a separate service class and data could be stored in a database.

7.4 DataFlow

User input → selected department and doctor → priority details → patient record creation → queue grouping and sorting → token/position/wait estimate → queue status table.

7.5ValidationandError Handling

- Check required fields before proceeding.
- Show a message if the patient name is blank.
- Require a priority choice before continuing from the priority frame.
- Validate age and phone number formats in PatientRegistrationFrame.
- Handle cancelled file selection and missing proof gracefully if proof upload is enabled.
- Prevent duplicate token generation if the user clicks Generate Token repeatedly by accident.

8. Results & Discussion

The expected result is a connected desktop workflow where users can move through the application's frames, register a patient, choose a department and doctor, select priority, generate a token, and view the relevant queue. The queue table provides a readable summary of the order and estimated waiting time

8.1 Testing Checklist

- Home button opens the login screen.
- Login validation prevents empty credentials if required by the implementation.
- Registration rejects missing required details.
- Department selection is passed correctly to doctor selection.
- Doctor selection is passed correctly to priority selection.
- Priority selection is required before token generation.
- Generate Token creates a record and displays a token.
- Priority sorting affects only the same department– doctor queue.
- Update Queue refreshes the table.
- Back buttons return to the previous frame without losing selected values.

8.2 Discussion

The prototype demonstrates the key GUI and sorting concepts. The most important design rule is that queue grouping must use both department and doctor, not only department or priority. Because the queue list is static and in memory, the application is appropriate for a single-user demonstration. Multi-user operation, persistence, concurrency, and secure handling of medical information would require additional design and testing.

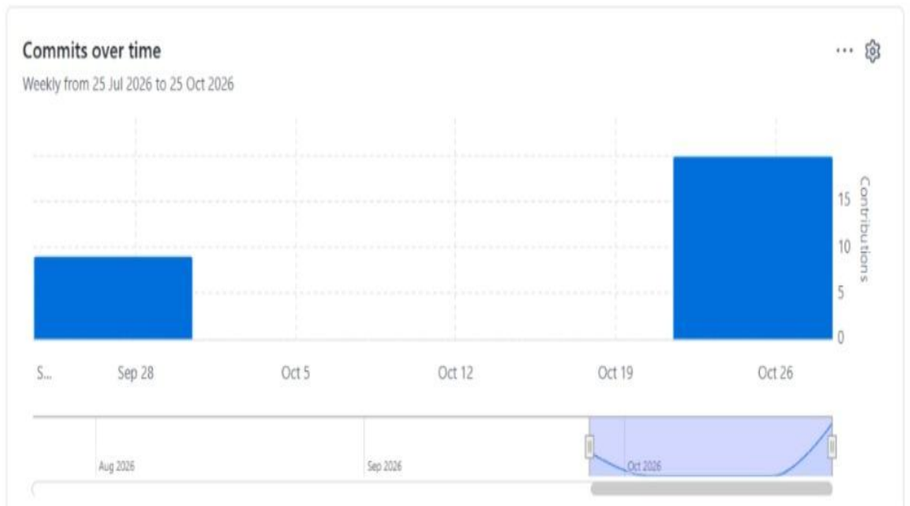
9. Conclusion

The Hospital Patient Token and Queue Management System demonstrates how Java Swing can be used to create a multi-frame application for patient registration and queue handling. It connects the Home, Login, Patient Registration, Department Selection, Doctor Selection, Priority Selection, Token Queue, and Queue Status screens. The queue algorithm separates patients by department and doctor and orders each group by priority and registration sequence. The project provides a useful practical exercise in event-driven programming, layout managers, collections, sorting, and table-based data presentation.

The current application is a prototype. Future work can add database integration, secure authentication, permanent patient records, more robust validation, and configurable waiting-time calculations

GITHUB PATH

https://github.com/nakshathra-d/Hospital-queue-management--system.git



References

1\. Oracle, Java Documentation. [https://docs.oracle.com/en/java/](https://docs.oracle.com/en/java/?utm_source=chatgpt.com)

2\. Oracle, The Java Tutorials – Creating a GUI with Swing. [https://docs.oracle.com/javase/tutorial/uiswing/](https://docs.oracle.com/javase/tutorial/uiswing/?utm_source=chatgpt.com)

3\. Oracle, Java SE API Documentation – javax.swing Package. [https://docs.oracle.com/en/java/javase/25/docs/api/java.desktop/javax/swing/package-summary.html](https://docs.oracle.com/en/java/javase/25/docs/api/java.desktop/javax/swing/package-summary.html?utm_source=chatgpt.com)

4 Gthub,hospital queue management system-project source code

https://github.com/nakshathra-d/Hospital-queue-management--system.git

tech//OOP/tree/main/src/hospital queue management